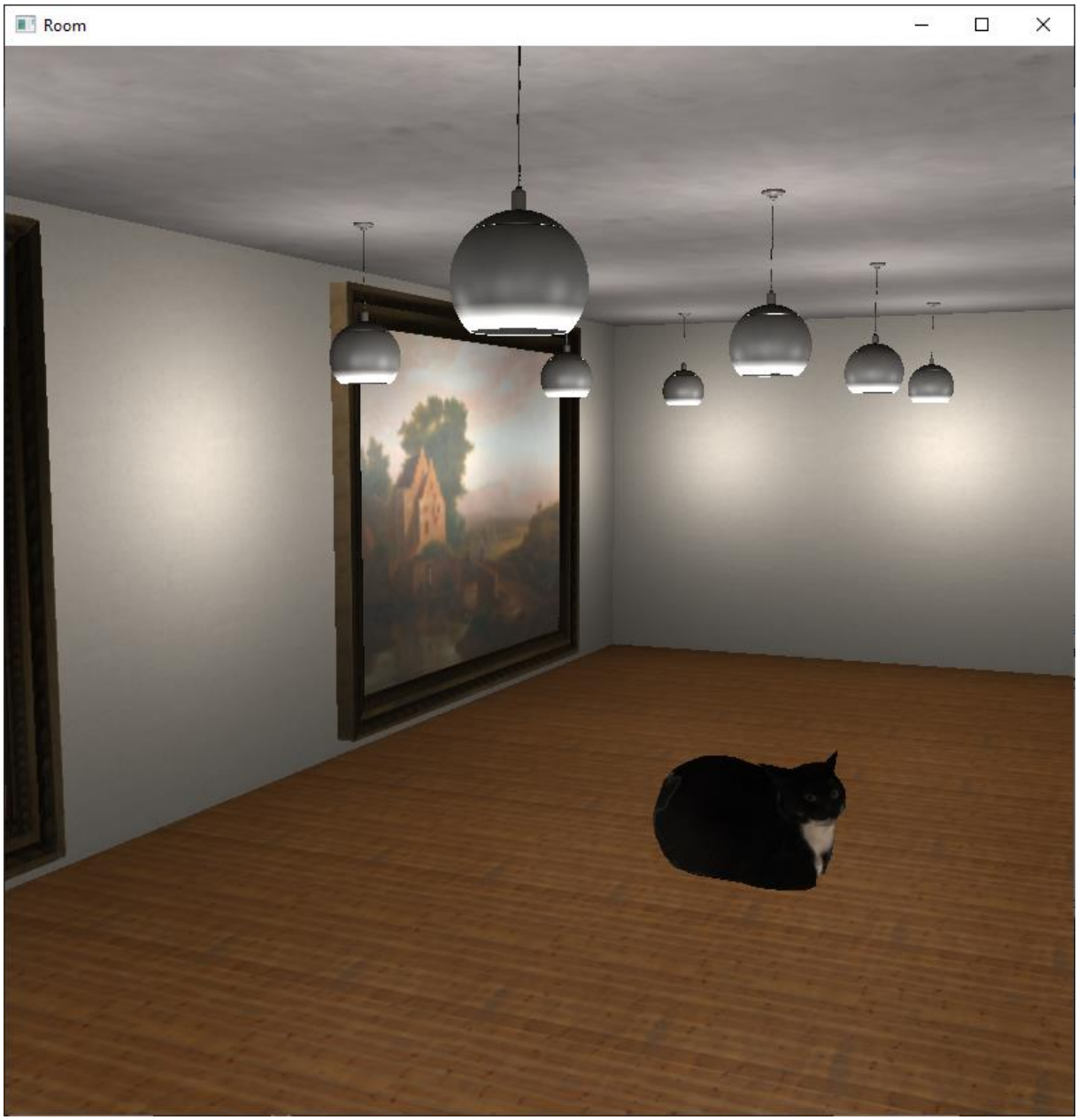
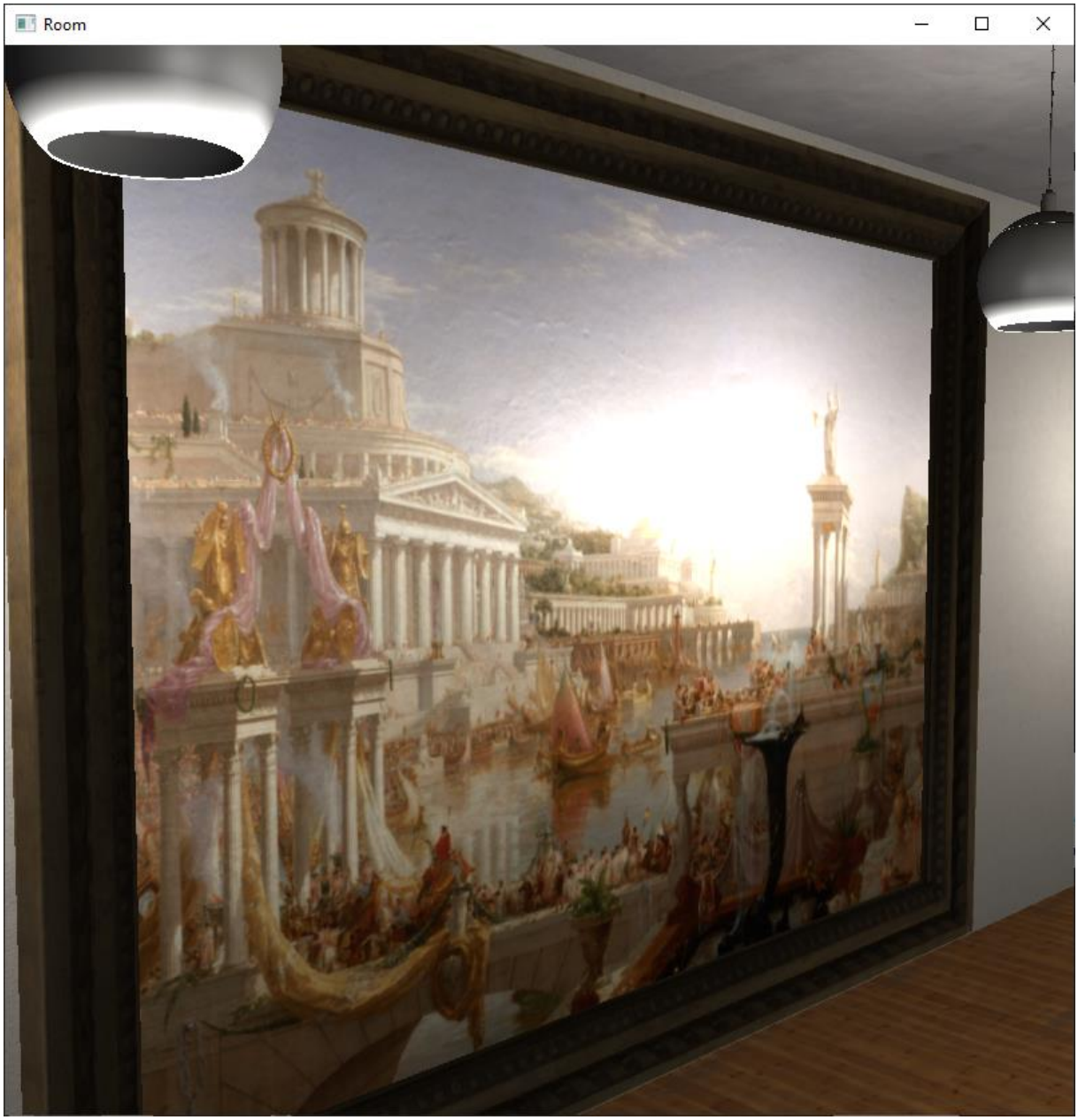
Sprawozdanie: Grafika komputerowa projekt zaliczeniowy – Mini Galeria Sztuki

Jako temat swojego projektu wybrałem „Mini Galerię sztuki”. Stworzony przeze mnie projekt przedstawia zamknięte pomieszczenie w którym znajdują się przykładowe obrazy oraz figurka kota, która kręci się wokół własnej osi. Została zaimplementowana możliwość poruszania się WASD po galerii sztuki Tekstury w wysokiej jakości (2k i 4k) nadały ładny wygląd całej galerii sztuki.

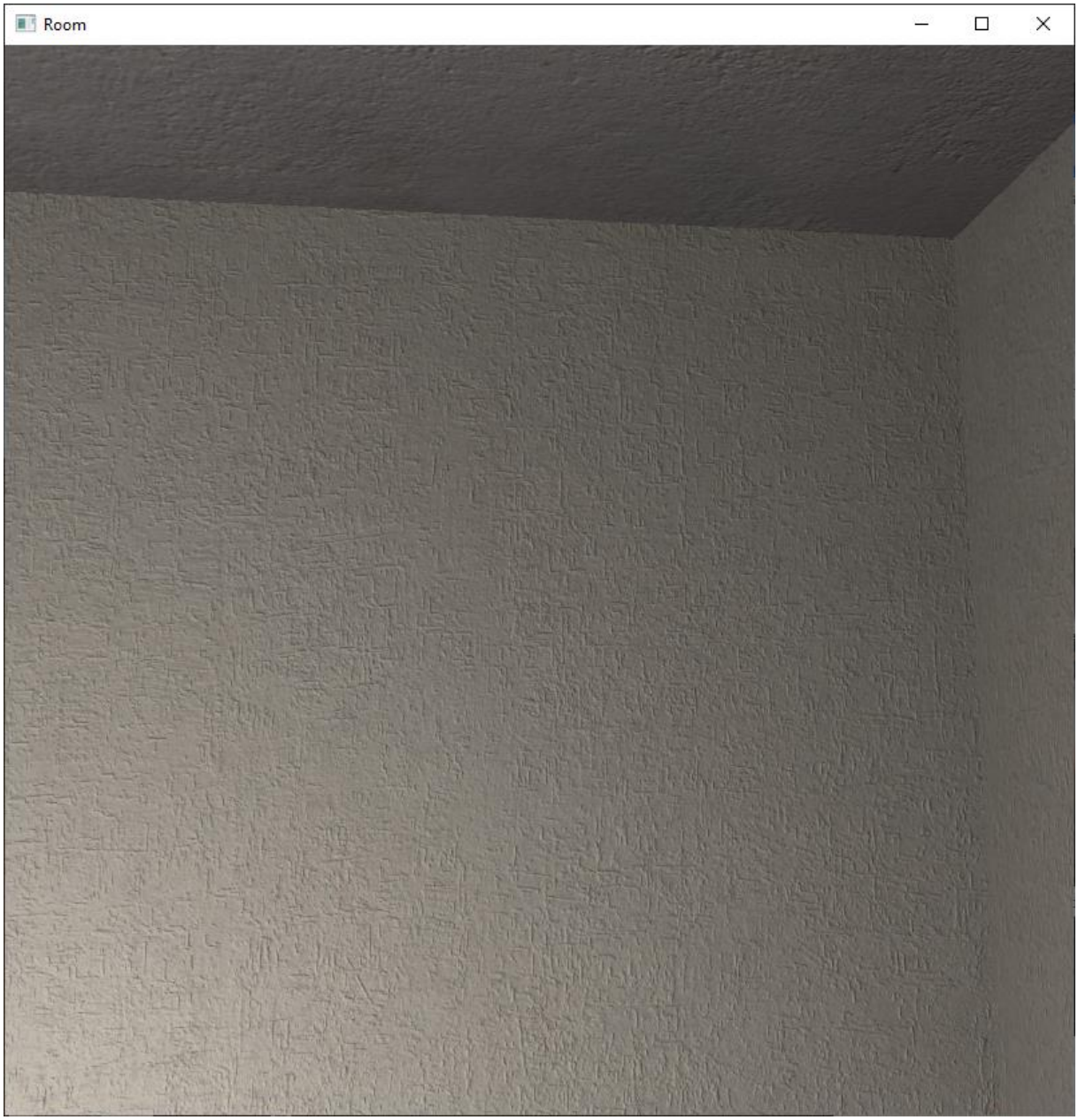


WASD - Poruszanie się
Przytrzymanie lewego przycisku myszy, umożliwia obrót kamerą za pomocą myszy
Shift – Przyśpieszenie ruchu
F – zmiana kolorystykę na sepię, G – zmiana na standardową kolorystykę

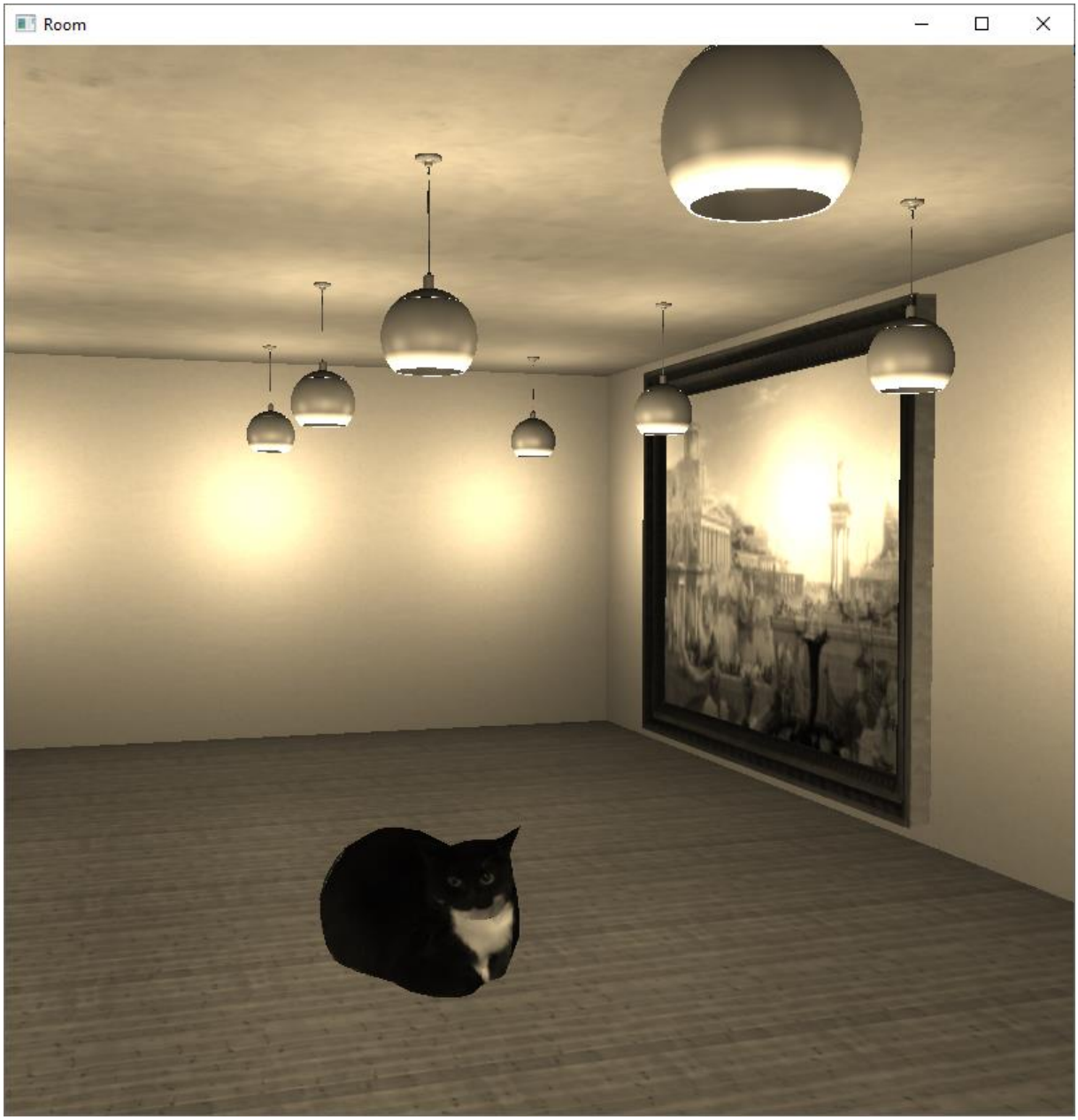
2. Obrazy są w wysokiej rozdzielczości oraz ich powłoka reaguje na światło oraz ma chropowatą teksturę, w celu większej imersji obrazu.



2. Ściany i sufit mają swoją chropowatą teksturę, na której widać cieniowanie, dzięki której można mieć wrażenie że tekstura jest 3D.



4. W programie został zaimplementowany jeden tryb filtrowania obrazu, w którym możemy mieć kolorystykę imitującą sepię.



5. Przykładowe fragmenty kodu

a. początek main.cpp zawierający include'y, *floorVerticies oraz ceilingVerticies*.

```
#include <iostream>
#include <glad/glad.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
#include "shaderClass.h"
#include "VAO.h"
#include "VBO.h"
#include "EBO.h"
#include "Camera.h"
#include "PBRMaterial.h"
#include "Model.h"

GLfloat floorVertices[] = {
    // posX, posY, posZ,  u, v,      nx, ny, nz,       tx, ty, tz,        bx, by, bz
    -15, -7.5, -30,  0.0f, 0.0f,     0, 1, 0,           1, 0, 0,           0, 0, 1,
     15, -7.5, -30, 10.0f, 0.0f,     0, 1, 0,           1, 0, 0,           0, 0, 1,
     15, -7.5,  30, 10.0f,10.0f,     0, 1, 0,           1, 0, 0,           0, 0, 1,
    -15, -7.5,  30,  0.0f,10.0f,     0, 1, 0,           1, 0, 0,           0, 0, 1,
};

GLfloat ceilingVertices[] = {
    -15, 7.5, -30, 0.0f, 0.0f,     0, -1, 0,     1, 0, 0,     0, 0, -1,
     15, 7.5, -30, 3.0f, 0.0f,     0, -1, 0,     1, 0, 0,     0, 0, -1,
     15, 7.5,  30, 3.0f, 6.0f,     0, -1, 0,     1, 0, 0,     0, 0, -1,
    -15, 7.5,  30, 0.0f, 6.0f,     0, -1, 0,     1, 0, 0,     0, 0, -1,
};
```

b. Rysowanie ramek oraz płótna z obrazami w głównej pętli programu:

```
if (frameModel && canvasModel) {
    for (int i = 0; i < 4; ++i) {
        float offsetZ = -14.9f + (i % 2) * 30.0f;
        float offsetY = 0.0f;
        float offsetX = (i < 2) ? -14.9f : 14.9f;
        float rotation = (i < 2) ? 90.0f : -90.0f;

        glm::mat4 frameMatrix = glm::mat4(1.0f);
        frameMatrix = glm::translate(frameMatrix, glm::vec3(offsetX, offsetY, offsetZ));
        frameMatrix = glm::rotate(frameMatrix, glm::radians(rotation), glm::vec3(0.0f, 1.0f, 0.0f));
        frameMatrix = glm::scale(frameMatrix, glm::vec3(30.0f));
        glUniformMatrix4fv(glGetUniformLocation(shader.ID, "model"), 1, GL_FALSE, glm::value_ptr(frameMatrix));

        glActiveTexture(GL_TEXTURE0); glBindTexture(GL_TEXTURE_2D, frameMat.albedo);
        glActiveTexture(GL_TEXTURE1); glBindTexture(GL_TEXTURE_2D, frameMat.normal);
        glActiveTexture(GL_TEXTURE2); glBindTexture(GL_TEXTURE_2D, frameMat.roughness);
        frameModel->Draw(shader);

        glActiveTexture(GL_TEXTURE0); glBindTexture(GL_TEXTURE_2D, paintingTextures[i]);
        glActiveTexture(GL_TEXTURE1); glBindTexture(GL_TEXTURE_2D, canvasMat.normal);
        glActiveTexture(GL_TEXTURE2); glBindTexture(GL_TEXTURE_2D, canvasMat.roughness);
        canvasModel->Draw(shader);
    }
}
```

c. funkcja main shaderów default.frag:

```
void main()
{
    vec3 albedo = texture(albedoMap, TexCoords).rgb;
    float roughness = clamp(texture(roughnessMap, TexCoords).r, 0.04, 1.0);

    mat3 TBN = mat3(normalize(T), normalize(B), normalize(N));
    vec3 normal = texture(normalMap, TexCoords).rgb;
    normal = normalize(normal * 2.0 - 1.0);
    normal = normalize(TBN * normal);


    vec3 viewDir = normalize(viewPos - FragPos);
    vec3 result = vec3(0.0);

    float lightPower = 20.0;
    vec3 lightColor = vec3(1.0);

    for (int i = 0; i < NUM_LIGHTS; ++i)
    {
        vec3 lightDir = normalize(lightPositions[i] - FragPos);
        float diff = max(dot(normal, lightDir), 0.0);

        vec3 halfway = normalize(lightDir + viewDir);
        float spec = pow(max(dot(normal, halfway), 0.0), mix(4.0, 64.0, 1.0 - roughness));

        float distance = length(lightPositions[i] - FragPos);
        float attenuation = lightPower / (1.0 + distance * distance);

        vec3 diffuse = diff * albedo * attenuation;
        vec3 specular = lightColor * spec * (1.0 - roughness) * attenuation;

        result += diffuse + specular;
    }

    vec3 ambient = 0.1 * albedo;
    vec3 finalColor = ambient + result;
    if (useSepia) {
        finalColor = vec3(
            dot(finalColor, vec3(0.393, 0.769, 0.189)),
            dot(finalColor, vec3(0.349, 0.686, 0.168)),
            dot(finalColor, vec3(0.272, 0.534, 0.131))
        );
    }
    FragColor = vec4(min(finalColor, vec3(1.0)), 1.0);

}
```